

Season 1: How LLMs work

Embeddings.

How models turn tokens into meaning they can compare

Meaning you can measure

- **Inside every model:** the first thing a model does with a token ID is look up its embedding
- **Search:** find text by what it means, not the words it uses
- **Retrieval:** RAG, in Season 2, is built on them
- **Grouping:** cluster, deduplicate, and classify without labels

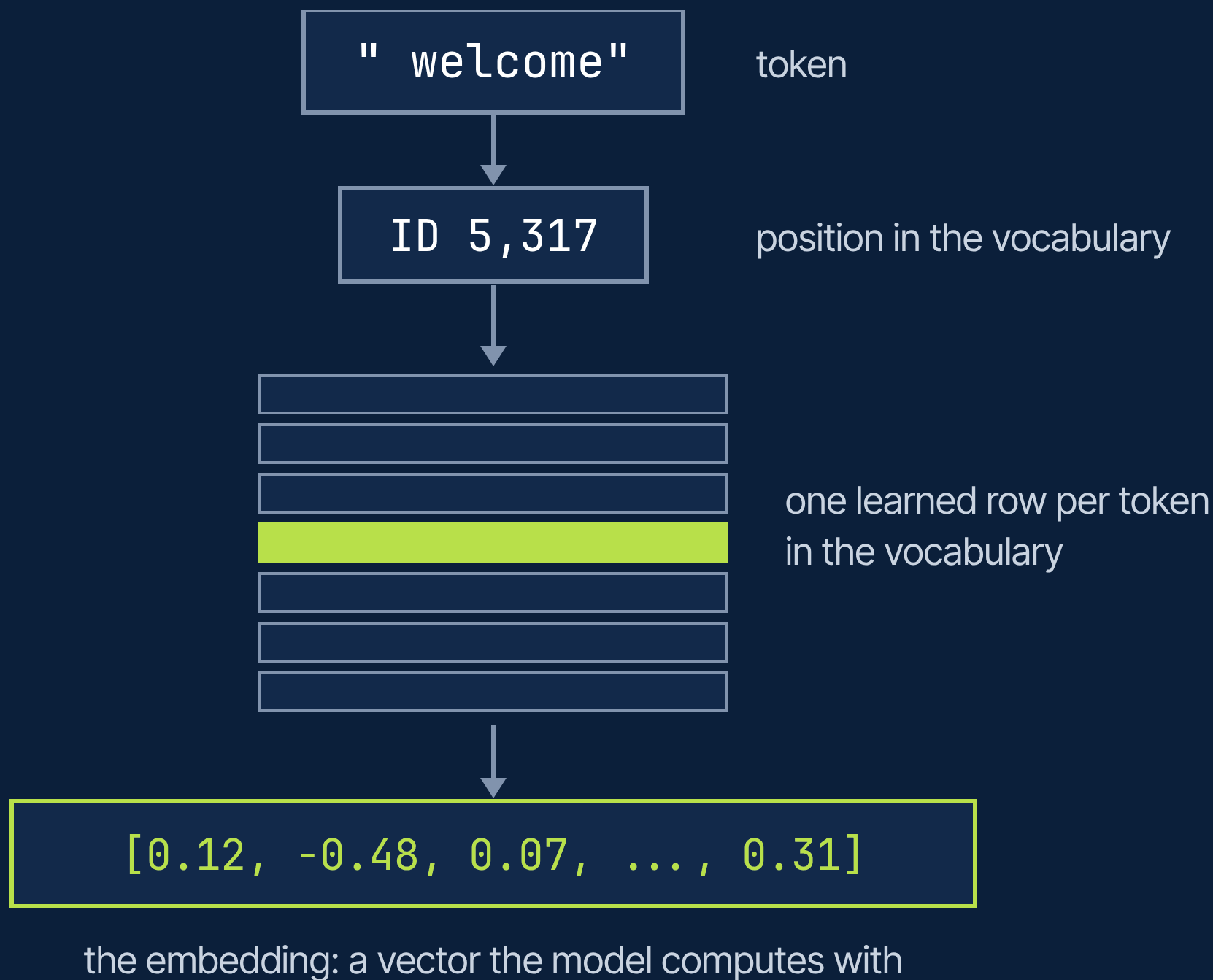
An embedding is a point in a space of meaning

A list of numbers: 1,536 of them for OpenAI's text-embedding-3-small. Texts with similar meaning sit close together.

Where it breaks

Close means similar, not correct. "The drug is safe" and "The drug is not safe" scored 0.807, closer than a true paraphrase at 0.592.

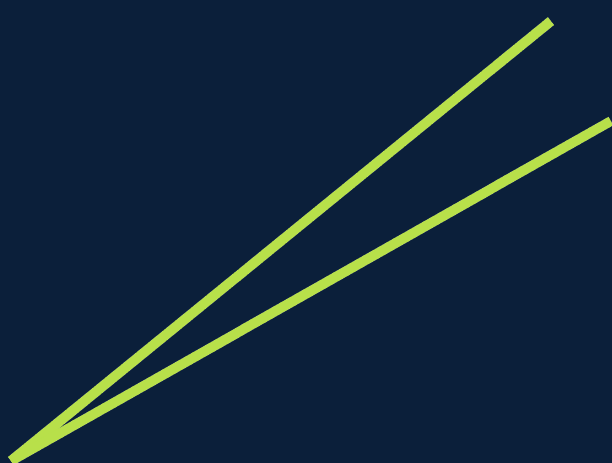
From token to vector



Illustrative ID and values. The table is learned in training, one row for each vocabulary entry.

Closeness is measured by angle

Cosine similarity compares the direction of two vectors, not their length.



Close to 1: same meaning



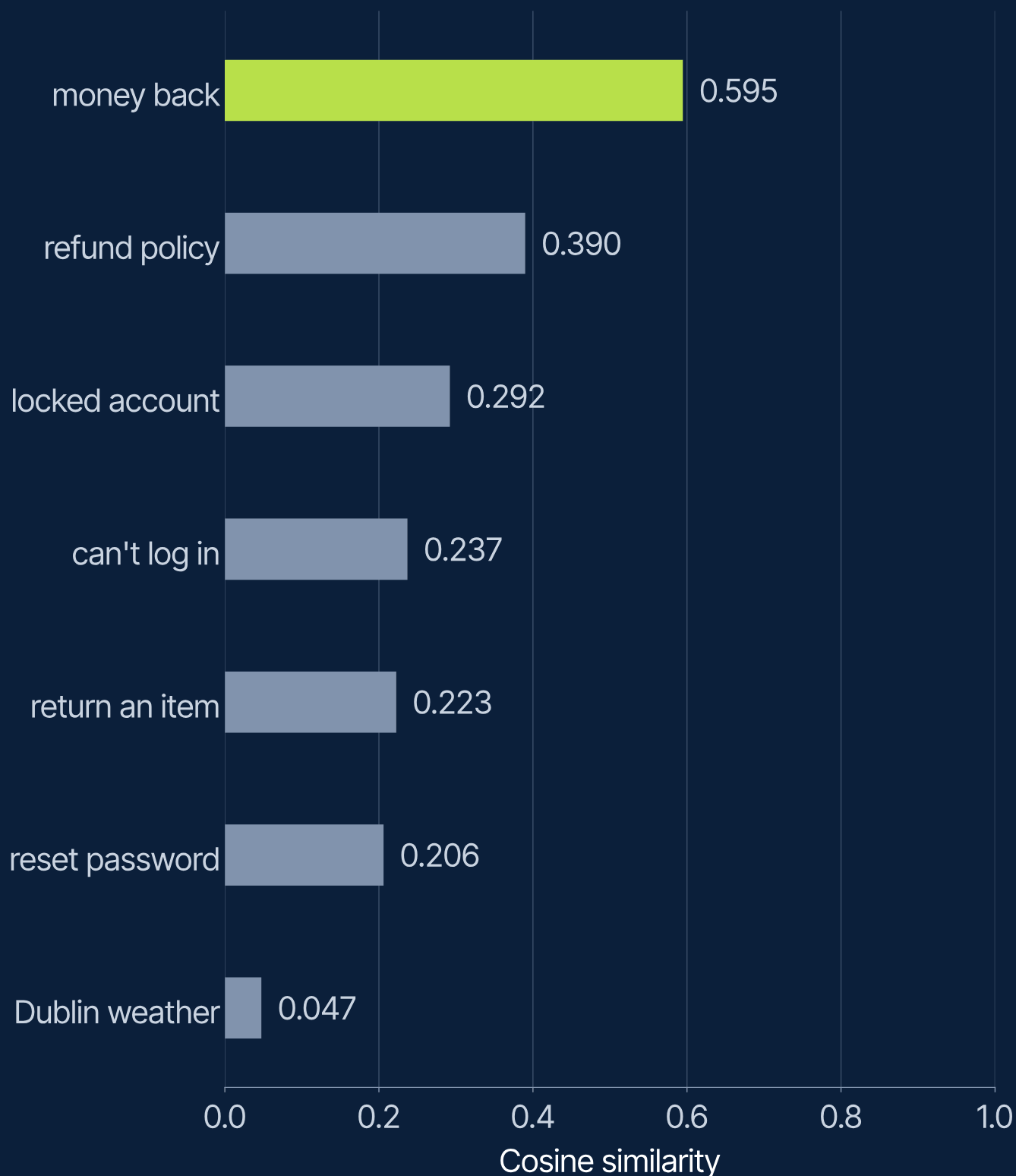
Close to 0: unrelated

Search embeds the question, then returns the stored passages whose vectors point the most nearly the same way.

Real scores sit in a narrow band: in this episode, unrelated sentences scored 0.104, not 0. Compare scores; do not read them as absolutes.

"refund policy" was found.
"return an item" was not.

Green: the top score, which shares two words with the query.



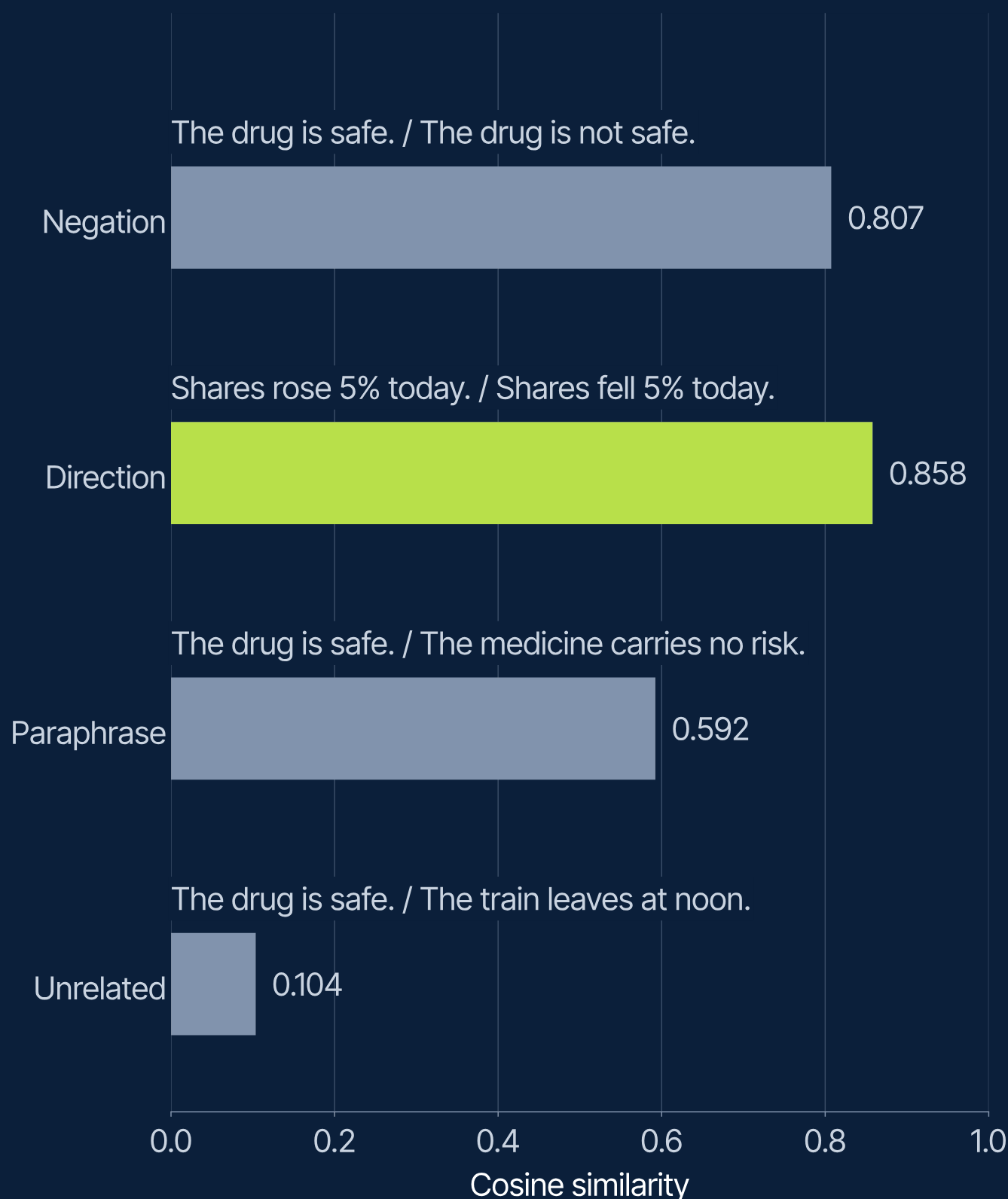
Model: text-embedding-3-small.

Cosine similarity to: How do I get my money back?

Cosine similarity · 7 stored phrases, one run

A sentence and its opposite scored closer than a paraphrase

Green: the closest opposite pair, shares that rose and fell.



Model: text-embedding-3-small.
Cosine similarity · 4 pairs, one run

Two kinds of embedding

Token embeddings

Inside the model: one vector per token, reshaped layer by layer as the model reads (S1 E4 and E5)

Text embeddings

From an embedding model: one vector for a whole passage, stored and searched (Season 2)

Where it breaks

Vectors from different models live in different spaces. They cannot be compared, even at the same length.

Similar is not the same as relevant

- **Opposites:** "safe" and "not safe" scored 0.807, above a paraphrase at 0.592
- **Direction:** "rose 5%" and "fell 5%" scored 0.858, the closest pair tested
- **Missed meaning:** for a refund question, "return an item" ranked below "can't log in"
- **Exact terms:** product codes, IDs, and names match poorly, which is why hybrid search adds keywords

text-embedding-3-small, one run, 19 September 2026.

Run it yourself

One API call embeds the query and every phrase.

```
import numpy as np
from openai import OpenAI

client = OpenAI()
query = "How do I get my money back?"
phrases = ["refund policy", "money back", "return an item",
           "reset password", "can't log in", "locked account",
           "Dublin weather"]
result = client.embeddings.create(
    model="text-embedding-3-small", input=[query, *phrases])
vectors = np.array([item.embedding for item in result.data])
q, rows = vectors[0], vectors[1:]
norms = np.linalg.norm(rows, axis=1) * np.linalg.norm(q)
scores = rows @ q / norms
for score, phrase in sorted(zip(scores, phrases), reverse=True):
    print(f"{score:.3f}  {phrase}")
```

■ Cosine similarity: the dot product, divided by both lengths

[episodes/s1-e3-embeddings/similarity.py](#). Needs an OpenAI API key; the charts redraw from committed results without one.

Working with vectors

One model

Embed queries and documents with the same model and version

Record it

Store the model name and version with every vector

Normalise

Scale vectors to length 1 if the provider does not, so dot product equals cosine

Test locally

Measure retrieval on your own queries before trusting a public leaderboard

What breaks in production

- **Silent upgrades:** a provider changes the model, and new queries no longer match the stored index
- **Mixed indexes:** vectors from two models in one store return nonsense rankings
- **Guessed thresholds:** a similarity cut-off chosen by eye drops good results or admits bad ones
- **Chunks that are too long:** one vector averages many ideas, and matches none of them well (Season 2)

Changing embedding model is a migration, not a setting

A new model cannot search an index built by the old one.

Switching means:

- **Re-embed everything:** every stored document, not just new content
- **Rebuild and retune:** a new index, and new thresholds for the new model's scores
- **Re-test retrieval:** on your own queries, before switching over

Ask your team

Do we keep the source text for every vector, and what would it take to switch embedding model?

Decision rule

Treat the embedding model as a versioned dependency: pin it, test it on your own queries, and re-embed when it changes.

Next: S1 E4 Attention

How a model decides which earlier tokens matter for the next one.

Where this fits

The series map

S1 LLMs	S2 Retrieval	S3 Strategy
S4 Adapting	S5 Agents	S6 Economics
S7 Data	S8 Evaluation	S9 AI teams
S10 Reliability	S11 Security	S12 Delivery
S13 Performance	S14 Resilience	S15 Governance

☐ Leadership season

 Follow	linkedin.com/in/brendan-kelly-irl
 Episode guide	brendankellyai.github.io
 Code	github.com/BrendanKellyAI/applied-ai-lab